

Table of Contents

The complete NumPy syllabus — foundations to full interview readiness

1

Introduction to NumPy
What is NumPy, why it exists, ndarray basics

\n

2

Creating Arrays
zeros, ones, full, eye, arange, linspace, random

\n

3

Array Attributes
shape, ndim, size, dtype, itemsize, astype

\n

4

Indexing & Slicing
1D/2D/3D indexing, slicing, boolean & fancy indexing

\n

5

Reshaping & Transposing
reshape, ravel, flatten, transpose, squeeze

\n

6

Copy vs View
Memory sharing, view(), copy(), base attribute

\n

7

Arithmetic & Broadcasting
Element-wise ops and broadcasting rules

\n

8

Universal Functions (ufuncs)
sqrt, exp, log, trig, vectorize

\n

9

Statistical & Aggregate Functions
sum, mean, std, argmin/argmax, axis

\n

10

Sorting, Searching & Set Ops
sort, argsort, where, unique, set ops

\n

11

Stacking & Splitting
concatenate, vstack/hstack, split

\n

12

Linear Algebra
dot, matmul, det, inverse, eig, solve

\n

13

Random Module Deep Dive

seed, shuffle, permutation, choice, distributions

\n

14

File I/O with NumPy

save/load, savez, savetxt/loadtxt

\n

15

Interview Questions

Most commonly asked NumPy interview Q&A

\n

16

Practice Questions

Hands-on exercises across the full syllabus

\n

17

Revision Cheat Sheet

One-page quick reference of every function

Introduction to NumPy

Understanding what NumPy is, why it matters, and the ndarray object at its core.

What is NumPy?

NumPy (Numerical Python) is an open-source Python library used for numerical computations, scientific computing, data analysis, and Machine Learning / Deep Learning. It provides a powerful object called **ndarray** (N-dimensional array), which is much faster and more memory-efficient than plain Python lists.

Why Do We Need NumPy?

Suppose you want to add two lists element-wise.

Using Python Lists

```
a = [1,2,3]
b = [4,5,6]

c = []
for i in range(len(a)):
    c.append(a[i] + b[i])

print(c)
```

OUTPUT
[5, 7, 9]

Using NumPy Arrays

```
import numpy as np

a = np.array([1,2,3])
b = np.array([4,5,6])

print(a + b)
```

OUTPUT
[5 7 9]

✂ Why it's faster

NumPy internally uses optimized C code under the hood, making vectorized operations significantly faster than Python loops.

Advantages of NumPy

Speed

Much faster than Python lists for numerical work.

Less Memory

Consumes significantly less memory than lists.

Mathematical Functions

Ships with hundreds of built-in math functions.

Matrix Operations

Purpose-built for linear algebra workflows.

Machine Learning Support

NumPy is the backbone of the Python data stack — used internally by Pandas, Scikit-Learn, TensorFlow, and PyTorch.

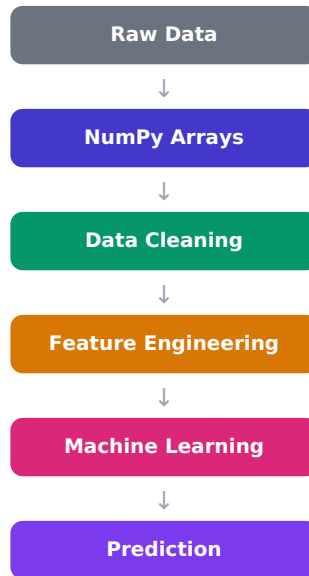
Installing NumPy

```
pip install numpy
```

Check version

```
import numpy as np
print(np.__version__)
```

NumPy in the Data Science Pipeline



What is ndarray?

An ndarray is a multidimensional array that can represent data in one, two, three, or more dimensions.

1D Array

```
[1, 2, 3, 4]
```

2D Array

```
[[1, 2],  
 [3, 4]]
```

3D Array

```
[  
  [[1,2],[3,4]],  
  [[5,6],[7,8]]  
]
```

Creating an Array

```
import numpy as np
arr = np.array([1,2,3,4])
print(arr)
```

OUTPUT
[1 2 3 4]

Creating 1D, 2D & 3D Arrays

1D Array

```
arr = np.array([1,2,3,4])
```

2D Array

```
arr = np.array([
    [1,2],
    [3,4]
])
```

3D Array

```
arr = np.array([
    [[1,2],[3,4]],
    [[5,6],[7,8]]
])
```

zeros() — Array Filled with Zeros

```
np.zeros(5)
```

OUTPUT

```
[0. 0. 0. 0. 0.]
```

2D version

```
np.zeros((3,4))
```

ones()

```
np.ones((2,3))
```

OUTPUT

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

full()

```
np.full((3,3), 7)
```

OUTPUT

```
[[7 7 7]
 [7 7 7]
 [7 7 7]]
```

eye() — Identity Matrix

```
np.eye(4)
```

OUTPUT

```
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
```

arange()

```
np.arange(1,10)
```

OUTPUT

```
[1 2 3 4 5 6 7 8 9]
```

```
np.arange(1,20,2)
```

OUTPUT

```
[1 3 5 7 9 11 13 15 17 19]
```

linspace()

```
np.linspace(0,10,5)
```

OUTPUT

```
[0.0 2.5 5.0 7.5 10.0]
```

🔗 arange vs linspace

arange() takes a step size; linspace() takes a count of evenly spaced points between start and stop (inclusive).

logspace()

```
np.logspace(1,3,5)
```

OUTPUT

```
[10.0, 31.62, 100.0, 316.2, 1000.0]
```

Random Arrays

```
np.random.rand(3)
np.random.randn(3)
np.random.randint(1,10,5)
```

📦 Quick distinction

rand() → uniform [0,1). randn() → standard normal distribution. randint() → random integers in a given range.

CHAPTER 03

Array Attributes

Inspecting shape, dimensions, size, data type and memory of any ndarray.

shape

```
arr = np.array([
    [1,2,3],
    [4,5,6]
])
print(arr.shape)
```

OUTPUT

```
(2, 3)
```

2 rows and 3 columns.

ndim

```
arr.ndim
```

OUTPUT

2

size

```
arr.size
```

OUTPUT

6

dtype

```
arr.dtype
```

OUTPUT

int64

itemsize

```
arr.itemsize
```

Memory occupied by one single element.

nbytes

```
arr.nbytes
```

Total memory occupied by the whole array.

astype()

```
arr.astype(float)
```

Full Example

```
arr = np.array([1,2,3])
print(arr.dtype)

new = arr.astype(float)
print(new.dtype)
```

⚠ Remember

shape → dimensions of the array. size → total number of elements. Easy to confuse in interviews!

CHAPTER 04

🔍 Indexing & Slicing

Accessing and extracting exactly the data you need from any dimension.

1D Indexing

```
arr = np.array([10,20,30,40])

print(arr[0])
print(arr[2])
print(arr[-1])
```

OUTPUT

```
10
30
40
```

2D Indexing

```
arr = np.array([
    [1,2,3],
    [4,5,6]
])

print(arr[0,1])
```

OUTPUT

```
2
```

3D Indexing

```
arr[0,1,0]
```

Slicing

```
arr = np.array([1,2,3,4,5,6])

arr[1:4]
```

OUTPUT

```
[2 3 4]
```

```
arr[:3]
```

OUTPUT

```
[1 2 3]
```

```
arr[::-2]
```

OUTPUT

```
[1 3 5]
```

2D Slicing

```
arr[:,1]
```

All rows and the second column.

```
arr[0,:]
```

First row.

```
arr[0:2,1:3]
```

OUTPUT

```
[[2 3]
 [5 6]]
```

Boolean Indexing

```
arr = np.array([1,2,3,4,5])

arr[arr > 3]
```


OUTPUT

```
[4 5]
```

Fancy Indexing

```
arr[[0,2,4]]
```

OUTPUT

```
[1 3 5]
```

🔗 Slicing vs Fancy Indexing

Slicing uses a continuous range (start:stop:step) and returns a view. Fancy indexing uses a list of specific indices and returns a copy.

NumPy Handbook • Chapter 4

CHAPTER 05

🔄 Reshaping & Transposing

Changing the shape of an array without changing its data.

reshape()

```
arr = np.arange(1,13)
new = arr.reshape(3,4)
print(new)
```

OUTPUT

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

👉 Rule of thumb

Total elements must stay the same before and after reshape (rows × columns = original size).

ravel() vs flatten()

```
arr = np.array([[1,2],[3,4]])
print(arr.ravel())
print(arr.flatten())
```

OUTPUT

```
[1 2 3 4]\n[1 2 3 4]
```

⚠️ Key difference

ravel() returns a view (shares memory with original) whenever possible. flatten() always returns a new copy.

transpose() / .T

```
arr = np.array([[1,2,3],[4,5,6]])
print(arr.T)
```

OUTPUT

```
[[1 4]
 [2 5]
 [3 6]]
```

resize()

```
arr = np.array([1,2,3,4])
arr.resize(2,2)
print(arr)
```

OUTPUT

```
[[1 2]
 [3 4]]
```

🔗 reshape() vs resize()

reshape() returns a new view without changing the original. resize() modifies the array in-place and can even pad or repeat data to fit a bigger shape.

newaxis & expand_dims()

```
arr = np.array([1,2,3])
print(arr[:, np.newaxis].shape)
print(np.expand_dims(arr, axis=0).shape)
```

OUTPUT

```
(3, 1)\n(1, 3)
```

squeeze()

```
arr = np.array([[[[1,2,3]]]])
print(arr.shape)
print(arr.squeeze().shape)
```

OUTPUT

```
(1, 1, 3)\n(3,)
```

Removes all axes of length 1 from the array.

NumPy Handbook • Chapter 5

CHAPTER 06

🔄 Copy vs View

Understanding whether an operation shares memory with the original array.

View — Shares Memory

```
arr = np.array([1,2,3,4])
v = arr.view()

v[0] = 100
print(arr)
```

OUTPUT

```
[100  2  3  4]
```

Changing the view also changes the original array — they share the same underlying data buffer.

Copy — Independent Memory

```
arr = np.array([1,2,3,4])
c = arr.copy()

c[0] = 100

print(arr)
print(c)
```

OUTPUT

```
[1 2 3 4]\n[100  2  3  4]
```

The original array is untouched because `copy()` allocates a completely new memory block.

How to Check: base attribute

```
arr = np.array([1,2,3,4])
v = arr.view()
c = arr.copy()

print(v.base is arr)
print(c.base is arr)
```

OUTPUT

```
True\nFalse
```

🔗 Interview tip

Slicing an ndarray always returns a VIEW, not a copy. Fancy indexing and boolean indexing always return a COPY.

NumPy Handbook • Chapter 6

CHAPTER 07

+ Arithmetic Operations & Broadcasting

Element-wise math and how NumPy handles arrays of different shapes.

Element-wise Arithmetic

```
a = np.array([1,2,3])
b = np.array([4,5,6])

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a ** 2)
```

OUTPUT

```
[5 7 9]\n[-3 -3 -3]\n[ 4 10 18]\n[0.25 0.4  0.5 ]\n[1 4 9]
```

What is Broadcasting?

Broadcasting lets NumPy perform arithmetic on arrays of different shapes by automatically "stretching" the smaller array without actually copying data.

```
arr = np.array([1,2,3])

print(arr + 10)
```

OUTPUT

```
[11 12 13]
```

Here the scalar 10 is broadcast across every element of the array.

Broadcasting with 2D Arrays

```
a = np.array([
    [1,2,3],
    [4,5,6]
])
b = np.array([10,20,30])
print(a + b)
```

OUTPUT

```
[[11 22 33]
 [14 25 36]]
```

The 1D array `b` is broadcast across each row of the 2D array `a`.

Broadcasting Rules

1 Rule 1

Compare shapes from the rightmost dimension moving left.

2 Rule 2

Two dimensions are compatible if they are equal, or one of them is 1.

3 Rule 3

If shapes are incompatible in any dimension (and neither is 1), NumPy raises a `ValueError`.

NumPy Handbook • Chapter 7

CHAPTER 08

⚙️ Universal Functions (ufuncs)

Fast, vectorized functions that operate element-by-element on arrays.

Common Mathematical ufuncs

```
arr = np.array([1,4,9,16])

print(np.sqrt(arr))
print(np.exp(arr[:2]))
print(np.log(arr))
print(np.abs(np.array([-1,-2,3])))
```

OUTPUT

```
[1.  2.  3.  4.]
[ 2.71828183  54.59815003]
[0.          1.38629436  2.19722458  2.77258872]
[1  2  3]
```

Trigonometric ufuncs

```
angles = np.array([0, np.pi/2, np.pi])

print(np.sin(angles))
print(np.cos(angles))
```

OUTPUT

```
[0.0000000e+00  1.0000000e+00  1.2246468e-16]
[ 1.0000000e+00  6.123234e-17 -1.0000000e+00]
```

ufuncs as Operators

```
a = np.array([1,2,3])
b = np.array([4,5,6])

print(np.add(a,b))
print(np.multiply(a,b))
print(np.greater(a,2))
```

OUTPUT

```
[5 7 9]\n[ 4 10 18]\n[False False  True]
```

✂ Why ufuncs matter

ufuncs are implemented in compiled C loops, so they avoid Python-level loops entirely — this is the real source of NumPy's speed advantage.

np.vectorize()

```
def my_func(x):
    return x*2 if x > 0 else 0

vec_func = np.vectorize(my_func)

print(vec_func(np.array([-1,2,-3,4])))
```

OUTPUT

```
[0 4 0 8]
```

Turns an ordinary Python function into one that applies element-wise over a NumPy array. Convenient, but not as fast as a true built-in ufunc.

NumPy Handbook • Chapter 8

CHAPTER 09

📊 Statistical & Aggregate Functions

Summarizing arrays with sums, averages, spreads and running totals.

sum, mean, median

```
arr = np.array([1,2,3,4,5])

print(np.sum(arr))
print(np.mean(arr))
print(np.median(arr))
```

OUTPUT

```
15\n3.0\n3.0
```

std & var

```
arr = np.array([1,2,3,4,5])

print(np.std(arr))
print(np.var(arr))
```

OUTPUT

```
1.4142135623730951\n2.0
```

min, max, argmin, argmax

```
arr = np.array([4,2,9,1])

print(np.min(arr), np.max(arr))
print(np.argmin(arr), np.argmax(arr))
```

```
OUTPUT
1 9\n3 2
```

`argmin()` / `argmax()` return the **index** of the minimum/maximum value, not the value itself.

cumsum & cumprod

```
arr = np.array([1,2,3,4])
print(np.cumsum(arr))
print(np.cumprod(arr))
```

```
OUTPUT
[ 1  3  6 10]\n[ 1  2  6 24]
```

The axis Parameter

```
arr = np.array([
    [1,2,3],
    [4,5,6]
])
print(np.sum(arr, axis=0))
print(np.sum(arr, axis=1))
```

```
OUTPUT
[5 7 9]\n[ 6 15]
```

⚠ axis=0 vs axis=1

`axis=0` collapses down the ROWS (operates column-wise). `axis=1` collapses across the COLUMNS (operates row-wise). This trips up almost everyone at first!

NumPy Handbook • Chapter 9

CHAPTER 10

🔍 Sorting, Searching & Set Operations

Ordering data, finding values, and comparing sets of elements.

sort()

```
arr = np.array([3,1,4,1,5,9,2])
print(np.sort(arr))
```

```
OUTPUT
[1 1 2 3 4 5 9]
```

argsort()

```
arr = np.array([3,1,2])
print(np.argsort(arr))
```

```
OUTPUT
[1 2 0]
```

Returns the indices that would sort the array, instead of the sorted values themselves.

where()

```
arr = np.array([1,2,3,4,5])

print(np.where(arr > 3))
print(np.where(arr > 3, "big", "small"))
```

OUTPUT

```
(array([3, 4]),)\n['small' 'small' 'small' 'big' 'big']
```

nonzero()

```
arr = np.array([0,3,0,5])

print(np.nonzero(arr))
```

OUTPUT

```
(array([1, 3]),)
```

unique()

```
arr = np.array([1,2,2,3,3,3])

print(np.unique(arr))
```

OUTPUT

```
[1 2 3]
```

Set Operations

```
a = np.array([1,2,3,4])
b = np.array([3,4,5,6])

print(np.intersect1d(a,b))
print(np.union1d(a,b))
print(np.setdiff1d(a,b))
```

OUTPUT

```
[3 4]\n[1 2 3 4 5 6]\n[1 2]
```

CHAPTER 11

🔗 Stacking & Splitting Arrays

Combining multiple arrays together, or breaking one array apart.

concatenate()

```
a = np.array([1,2,3])
b = np.array([4,5,6])

print(np.concatenate((a,b)))
```

OUTPUT

```
[1 2 3 4 5 6]
```

vstack() & hstack()

```
a = np.array([1,2,3])
b = np.array([4,5,6])

print(np.vstack((a,b)))
print(np.hstack((a,b)))
```

```
OUTPUT
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
```

stack()

```
a = np.array([1,2,3])
b = np.array([4,5,6])

print(np.stack((a,b), axis=0))
```

```
OUTPUT
[[1 2 3]
 [4 5 6]]
```

⚡ stack() vs vstack()

stack() creates a brand-new dimension, while vstack() joins arrays along an existing axis.

split()

```
arr = np.arange(9)

print(np.split(arr, 3))
```

```
OUTPUT
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
```

hsplit() & vsplit()

```
arr = np.arange(16).reshape(4,4)

print(np.hsplit(arr, 2))
```

```
OUTPUT
[array of left 2 columns, array of right 2 columns]
```

CHAPTER 12

Linear Algebra with NumPy

Matrix multiplication, determinants, inverses and solving equations.

Matrix Multiplication: dot() / @

```
a = np.array([[1,2],[3,4]])
b = np.array([[5,6],[7,8]])

print(np.dot(a,b))
print(a @ b)
```

```
OUTPUT
[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
```

⚠ Important distinction

`a * b` multiplies element-wise. `a @ b` (or `np.dot/np.matmul`) performs true matrix multiplication. Mixing these up is a classic bug.

Transpose

```
a = np.array([[1,2],[3,4]])
print(a.T)
```

OUTPUT

```
[[1 3]
 [2 4]]
```

Determinant

```
a = np.array([[1,2],[3,4]])
print(np.linalg.det(a))
```

OUTPUT

```
-2.0000000000000004
```

Inverse

```
a = np.array([[1,2],[3,4]])
print(np.linalg.inv(a))
```

OUTPUT

```
[[-2.   1. ]
 [ 1.5 -0.5]]
```

Eigenvalues & Eigenvectors

```
a = np.array([[2,0],[0,3]])
values, vectors = np.linalg.eig(a)
print(values)
print(vectors)
```

OUTPUT

```
[2. 3.]
[[1. 0.]
 [0. 1.]]
```

Solving Linear Equations

```
# 2x + y = 5
# x + 3y = 10
A = np.array([[2,1],[1,3]])
b = np.array([5,10])
print(np.linalg.solve(A,b))
```

OUTPUT

```
[1. 3.]
```

CHAPTER 13

Random Module Deep Dive

Generating reproducible random data and sampling from distributions.

seed() — Reproducibility

```
np.random.seed(42)
print(np.random.rand(3))
```

OUTPUT

```
[0.37454012 0.95071431 0.73199394]
```

🔗 Why seed matters

Setting a seed makes 'random' results reproducible — essential for debugging ML experiments and sharing results.

shuffle()

```
arr = np.array([1,2,3,4,5])
np.random.shuffle(arr)
print(arr)
```

OUTPUT

```
[3 1 5 2 4] # shuffled in-place
```

permutation()

```
arr = np.array([1,2,3,4,5])
print(np.random.permutation(arr))
```

OUTPUT

```
[2 4 1 5 3]
```

Unlike `shuffle()`, `permutation()` returns a new shuffled copy instead of modifying the original array.

choice()

```
print(np.random.choice([10,20,30,40], size=2, replace=False))
```

OUTPUT

```
[20 40]
```

Common Distributions

```
print(np.random.normal(loc=0, scale=1, size=3))
print(np.random.uniform(low=0, high=10, size=3))
print(np.random.binomial(n=10, p=0.5, size=3))
```

OUTPUT

```
[ 0.49671415 -0.1382643  0.64768854]\n[6.34 1.51 9.02]\n[5 6 4]
```

CHAPTER 14

File I/O with NumPy

Saving and loading arrays to and from disk.

save() & load() — Binary .npy

```
arr = np.array([1,2,3])
np.save("my_array.npy", arr)
loaded = np.load("my_array.npy")
print(loaded)
```

OUTPUT

```
[1 2 3]
```

savez() — Multiple Arrays in .npz

```
a = np.array([1,2,3])
b = np.array([4,5,6])
np.savez("data.npz", first=a, second=b)
data = np.load("data.npz")
print(data["first"])
print(data["second"])
```

OUTPUT

```
[1 2 3]\n[4 5 6]
```

savetxt() & loadtxt() — Text / CSV

```
arr = np.array([[1,2],[3,4]])
np.savetxt("data.csv", arr, delimiter=",")
loaded = np.loadtxt("data.csv", delimiter=",")
print(loaded)
```

OUTPUT

```
[[1. 2.]
 [3. 4.]]
```

🔗 .npy vs .csv

.npy/.npz preserve exact dtype and shape and load faster. .csv/.txt are human-readable and portable across tools like Excel or Pandas.

CHAPTER 15



Important Interview Questions

The most frequently asked NumPy questions in Data Science & ML interviews.

Q1. Difference between a Python list and a NumPy array?

- NumPy arrays are faster for numerical operations.
- NumPy arrays consume less memory.
- NumPy arrays support vectorized operations (no explicit loops needed).

Q2. Difference between shape and size?

- **shape** → the dimensions of the array (rows, columns, ...).
- **size** → the total number of elements in the array.

Q3. Difference between slicing and fancy indexing?

- Slicing uses a start:stop:step range and returns a view.
- Fancy indexing uses an explicit list of indices and returns a copy.

Q4. Why is NumPy faster than plain Python?

- It uses an optimized C implementation internally.
- Array elements are stored in contiguous memory blocks, which is cache-friendly.

Q5. Difference between copy() and view()?

- view() shares the same underlying memory as the original array.
- copy() allocates a completely new, independent memory block.

Q6. What is Broadcasting in NumPy?

- A mechanism that lets NumPy perform arithmetic on arrays of different shapes.
- The smaller array is virtually "stretched" to match the larger array's shape without copying data.

Q7. What is a ufunc?

- A universal function that operates element-wise on ndarrays.
- Implemented in compiled C, so it's much faster than an equivalent Python loop.

Q8. Difference between axis=0 and axis=1?

- axis=0 operates down the rows (column-wise aggregation).
- axis=1 operates across the columns (row-wise aggregation).

Q9. Difference between np.vstack() and np.concatenate()?

- vstack() always stacks along a new/existing vertical axis and is a convenience wrapper.
- concatenate() is the general-purpose function and needs an explicit axis argument for 2D+ arrays.

Q10. Difference between element-wise multiplication and matrix multiplication?

- `a * b` multiplies corresponding elements.
- `a @ b` or `np.dot(a, b)` performs true matrix multiplication (dot product of rows and columns).

CHAPTER 16

Practice Questions

Test yourself across the full syllabus before your interview.

- 1 Create a 3×3 matrix of zeros.
- 2 Create an identity matrix of size 5.
- 3 Generate numbers from 1 to 100.
- 4 Create a matrix of random integers.
- 5 Find the shape and size of a matrix.

- 6 Extract the last three elements using slicing.
- 7 Extract even numbers using boolean indexing.
- 8 Reshape a 1D array of 12 elements into a 3×4 matrix.
- 9 Demonstrate the difference between `ravel()` and `flatten()` with an example.
- 10 Show that slicing returns a view while fancy indexing returns a copy.
- 11 Add a (3,) array to a (2,3) array and explain the broadcasting that occurs.
- 12 Use a ufunc to compute the square root of every element in an array.
- 13 Compute the column-wise and row-wise mean of a 2D array using the `axis` parameter.
- 14 Sort an array and separately return the sorting indices with `argsort()`.
- 15 Stack two 1D arrays vertically and horizontally.
- 16 Compute the matrix product of two 2×2 matrices using both `@` and `np.dot()`.
- 17 Set a random seed and generate the same 5 random numbers twice.
- 18 Save an array to a `.npy` file and load it back.

CHAPTER 17

Revision Cheat Sheet

One-page quick reference for last-minute revision, covering the full syllabus.

Function / Attribute	Purpose
<code>np.array()</code>	Create an array from a list/tuple
<code>np.zeros()</code>	Array filled with zeros
<code>np.ones()</code>	Array filled with ones
<code>np.full()</code>	Array filled with a constant value
<code>np.eye()</code>	Identity matrix
<code>np.arange()</code>	Range of values with a step size
<code>np.linspace()</code>	Evenly spaced values, given a count
<code>np.logspace()</code>	Evenly spaced values on a log scale
<code>arr.shape</code>	Dimensions of the array
<code>arr.size</code>	Total number of elements
<code>arr.ndim</code>	Number of dimensions
<code>arr.dtype</code>	Data type of elements
<code>arr.nbytes</code>	Total memory occupied

<code>arr.astype()</code>	Convert to another data type
<code>arr[1:5]</code>	Slice a range of elements
<code>arr[:,1]</code>	All rows, one column
<code>arr[arr>5]</code>	Boolean indexing
<code>arr[[0,2,4]]</code>	Fancy indexing
<code>arr.reshape()</code>	Change shape without changing data
<code>arr.ravel()</code>	Flatten to 1D, returns a view
<code>arr.flatten()</code>	Flatten to 1D, returns a copy
<code>arr.T / arr.transpose()</code>	Transpose rows and columns
<code>arr.resize()</code>	Change shape in-place
<code>np.expand_dims()</code>	Add a new axis
<code>arr.squeeze()</code>	Remove axes of length 1
<code>arr.view()</code>	New array object, same memory
<code>arr.copy()</code>	New array object, new memory
<code>a + b, a - b, a * b, a / b</code>	Element-wise arithmetic
Broadcasting	Arithmetic across different shapes
<code>np.sqrt(), np.exp(), np.log()</code>	Common ufuncs
<code>np.vectorize()</code>	Vectorize a plain Python function
<code>np.sum() / np.mean() / np.median()</code>	Aggregate statistics
<code>np.std() / np.var()</code>	Spread of the data
<code>np.argmin() / np.argmax()</code>	Index of min/max value
<code>np.cumsum() / np.cumprod()</code>	Running total / running product
<code>axis=0 / axis=1</code>	Column-wise vs row-wise aggregation
<code>np.sort() / np.argsort()</code>	Sort values / get sort indices
<code>np.where()</code>	Conditional selection
<code>np.unique()</code>	Distinct elements
<code>np.intersect1d/union1d/setdiff1d</code>	Set operations
<code>np.concatenate()</code>	Join arrays along an axis
<code>np.vstack() / np.hstack()</code>	Stack vertically / horizontally
<code>np.split() / np.hsplit() / np.vsplit()</code>	Break an array apart
<code>np.dot() / @ / np.matmul()</code>	Matrix multiplication
<code>np.linalg.det()</code>	Determinant of a matrix
<code>np.linalg.inv()</code>	Inverse of a matrix
<code>np.linalg.eig()</code>	Eigenvalues & eigenvectors
<code>np.linalg.solve()</code>	Solve a system of linear equations
<code>np.random.seed()</code>	Reproducible random numbers
<code>np.random.shuffle() / permutation()</code>	Shuffle in-place / return new copy
<code>np.random.choice()</code>	Random sample from an array
<code>np.save() / np.load()</code>	Save/load a single array (.npy)
<code>np.savez()</code>	Save multiple arrays (.npz)
<code>np.savetxt() / np.loadtxt()</code>	Save/load as text or CSV

★ **Key Takeaway**

This handbook covers the complete NumPy syllabus tested in Data Science and Machine Learning interviews — from array basics all the way to linear algebra and file I/O.